

12 | 数据库锁：明明有行锁，怎么突然就加了表锁？

2023-7-12 邓明



你好，我是大明。今天我们来聊一聊 MySQL 中锁的问题。

锁在整个数据库面试中都是属于偏难，而且偏琐碎的一类问题。但是偏偏锁又很重要，比如说实践中遇到死锁影响了性能，这都要求我们必须对锁有一定的了解。并且锁的原理和索引、隔离级别都有关，所以很容易从锁这个角度联想到另外两个地方，又或者从索引和隔离级别里面跳到锁这里。

因此，一句话总结就是**锁既难又琐碎还热门**。那么今天这节课我会带你彻底捋清楚锁，并且告诉你在面试过程中如何展示出你的亮点。

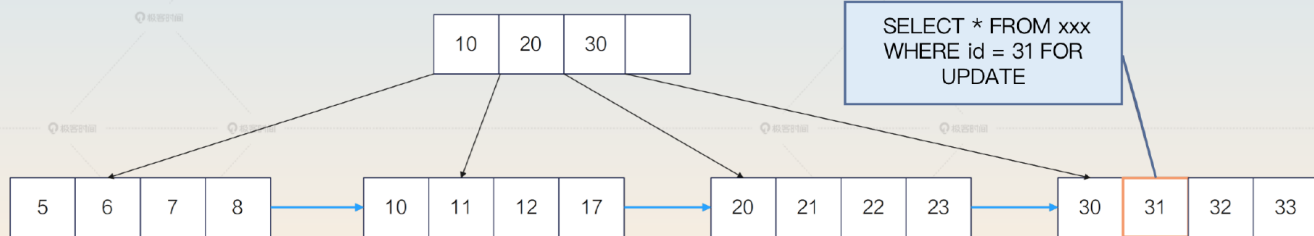
前置知识

锁与索引

在 MySQL 的 InnoDB 引擎里面，锁是借助索引来实现的。或者说，加锁锁住的其实是索引项，更加具体地来说，就是锁住了**叶子节点**。



id=31 的查询会锁住索引项 31



极客时间

你从这个角度出发，就能理解大部分跟锁有关的千奇百怪的问题了。

一个表有很多索引，锁的是哪个索引呢？其实就是查询最终使用的那个索引。万一查询没有使用任何索引呢？那么锁住的就是整个表，也就是此时退化为表锁。

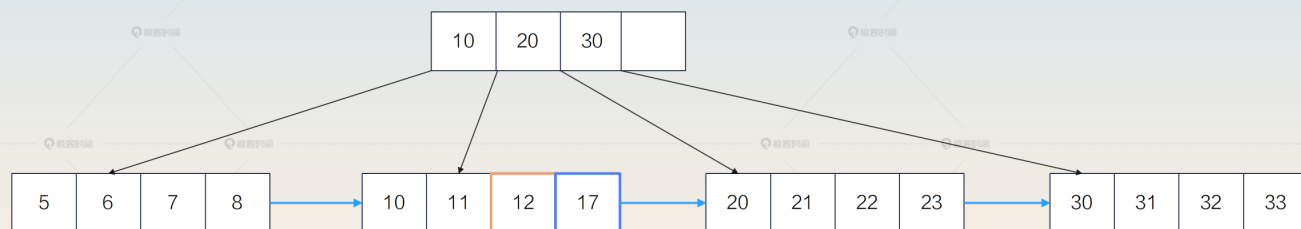
如果查询条件的值并不存在，例如：

```
SELECT * FROM your_tab WHERE id = 15 FOR UPDATE
```

id = 15 的值根本不存在，那么怎么锁？InnoDB 引擎会利用最接近 15 的相邻的两个节点，构造一个临键锁。



查询会锁住[12, 17]



因为 15 这个记录不存在，所以最终锁住的是 (12, 17]

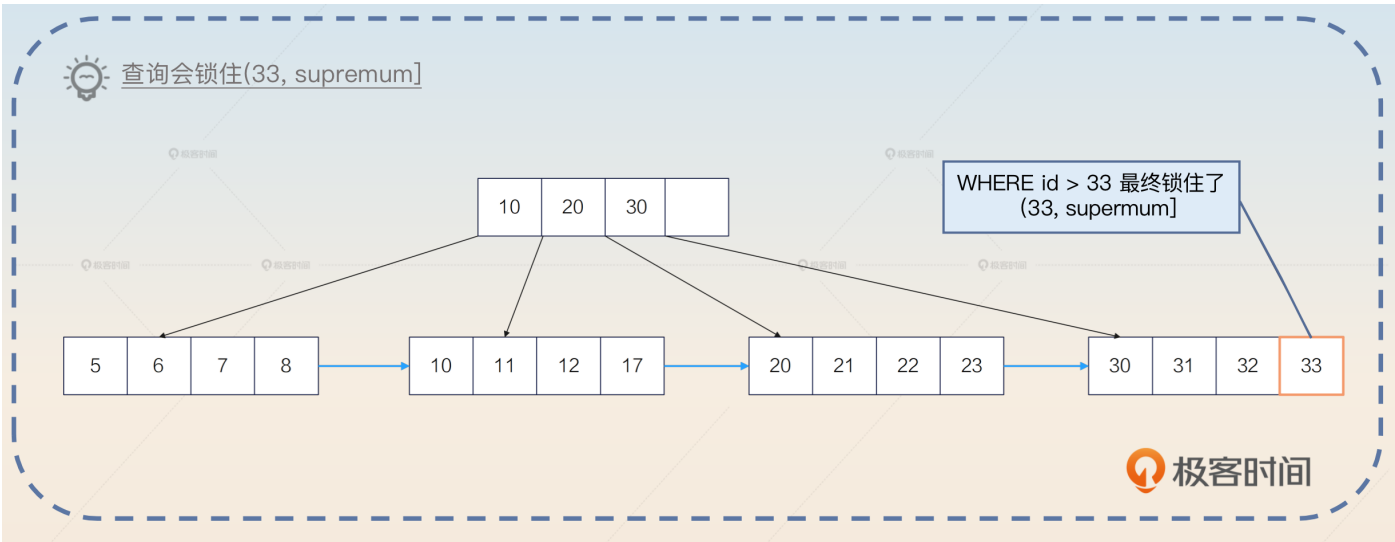
极客时间

此时如果别的事务想要插入一个 id=15 的记录，就不会成功。

那么范围查询呢？也是利用索引上的数据，构造一个恰好能够装下这个范围的临键锁。例如：

```
SELECT * FROM your_tab WHERE id > 33 FOR UPDATE
```

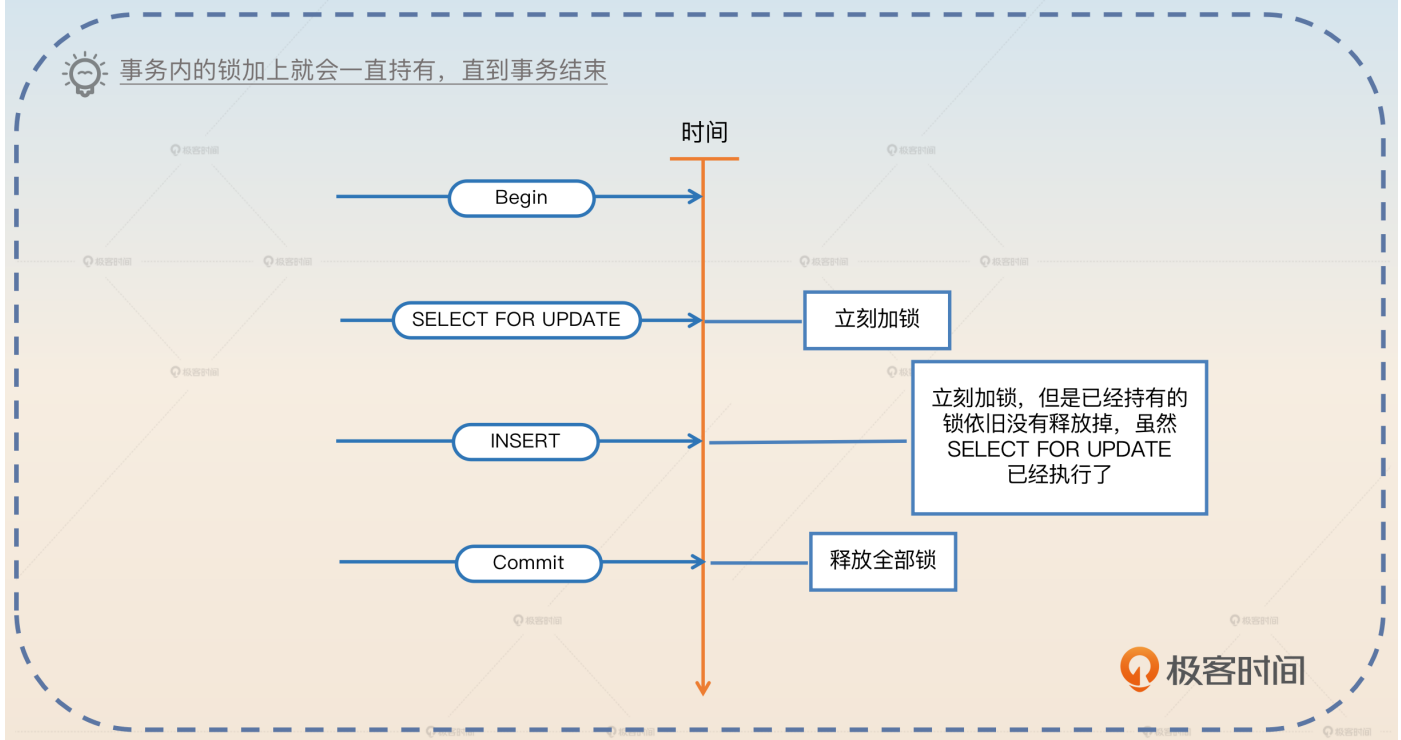
InnoDB 引擎会构造一个(33, supremum] 的临键锁，锁住整个范围。supremum 你可以直观理解为 MySQL 认为的一个虚拟的最大值。



因此，我们得出了一个结论：**锁和索引密切相关。**

释放锁时机

大部分人在学习锁的时候有一个误区，就是认为锁是在语句执行完毕之后就立刻释放掉的。其实并不是，它是在整个事务结束之后才释放的。换句话说，当一个事务内部给数据加上锁之后，**只有在执行 Rollback 或者 Commit 的时候，锁才会被释放掉。**



乐观锁与悲观锁

乐观锁和悲观锁实际上是一种逻辑概念，它们是并发控制中常用的两种锁机制。

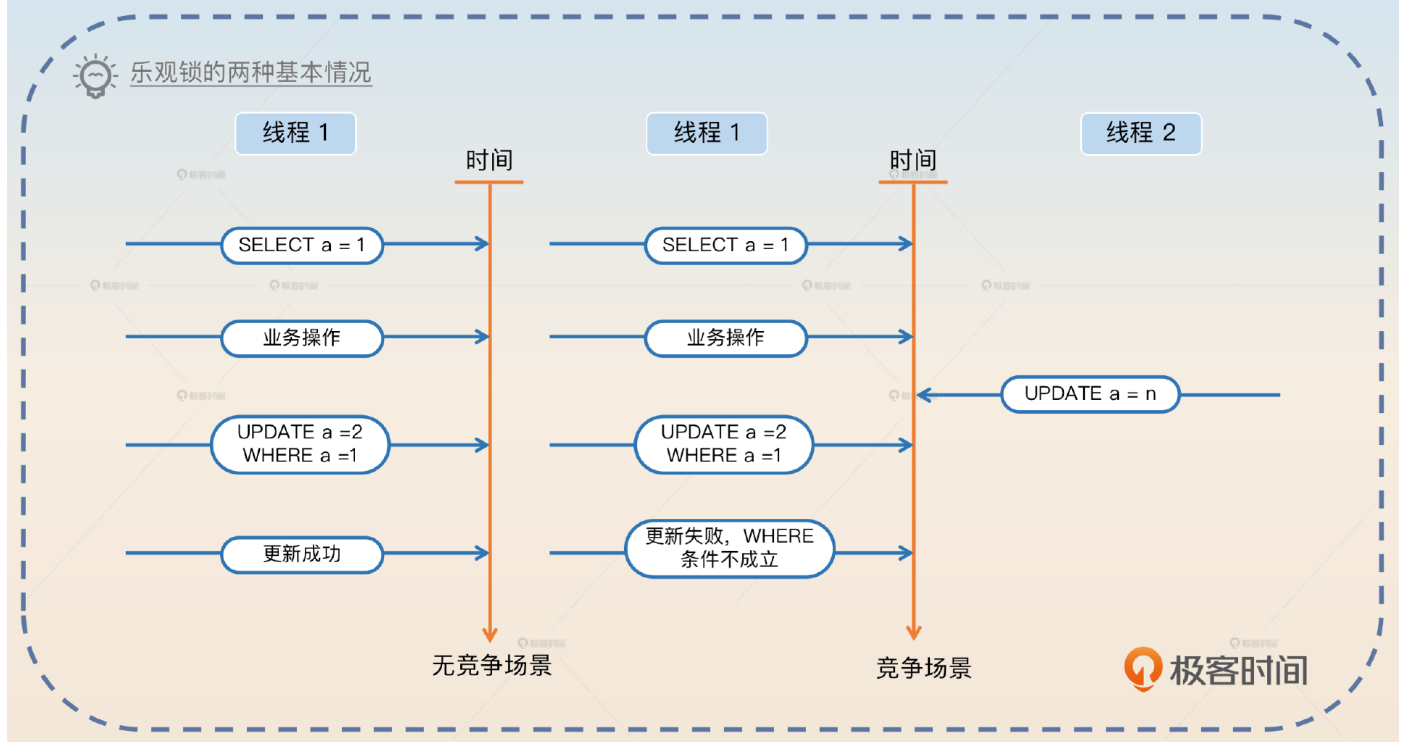
乐观锁是直到要修改数据的时候，才检测数据是否已经被别人修改过。

悲观锁是在初始时刻就直接加锁保护好临界资源。

乐观锁在数据库中通常指利用 CAS 的思路进行的更新操作。一般的使用形态就是下面这样的。

```
SELECT * FROM your_tab WHERE id = 1; // 在这里拿到了 a = 1
// 一大堆的业务操作
UPDATE your_tab SET a = 3, b = 4 WHERE id = 1 AND a = 1
```

在上面的这个语句里面，预期数据库中 a 的值为 1 才会进行更新。如果此时数据库中的值已经被修改了，那么这个 UPDATE 语句就会失败。业务方通过检测受影响行数是否为 0，来判断更新是否成功。



悲观锁是指在写入数据时直接加锁。还拿上面这个例子来说吧，就是从最开始的 SELECT 语句就直接加上了锁。

```
SELECT * FROM your_tab WHERE id = 1 FOR UPDATE; // 在这里拿到了 a = 1
// 一大堆的业务操作
UPDATE your_tab SET a = 3, b = 4 WHERE id = 1
```

在加上锁之后，就可以直接更新了。这个时候不需要担心别人可以在 SELECT 和 UPDATE 之间将 a 更新为别的值。

在使用乐观锁和悲观锁时，需要考虑数据一致性和并发性的问题。乐观锁适用于读多写少的场景，互联网中大部分应用都属于这一类。而悲观锁则适用于写多读少的场景，比如在金融领域里面对金额的操作就是以写为主。

相比之下，乐观锁的性能要比悲观锁好很多。不过因为乐观锁的代码写起来比较复杂，所以很多人偷懒就会直接使用悲观锁。

行锁与表锁

行锁与表锁都是根据锁的范围来划分的。一般来说，行锁是指锁住行，可能是锁住一行，也可能是锁住多行。表锁则是直接将整个表都锁住。

那么在 MySQL 里面，InnoDB 引擎同时支持行锁和表锁。但是行锁是借助索引来实现的，也就是说，如果你的查询没有命中任何的索引，那么 InnoDB 引擎是用不了行锁的，只能使用表锁。当然，如果用的是 MySQL，类似于 MyISAM 引擎，那么只能使用表锁，因为这些引擎不支持行锁。

共享锁与排它锁

共享锁和排它锁是在互斥的角度上看待锁的。

共享锁是指一个线程加锁之后，其他线程还是可以继续加同类型的锁。

排它锁是指一个线程加锁之后，其他线程就不能再加锁了。

类型	共享锁	排它锁
共享锁	兼容	不兼容
排它锁	不兼容	不兼容

这两个概念非常接近读锁和写锁。因为读锁本身就是共享的，而写锁就是排它的。

意向锁

意向锁相当于一个信号，就是告诉别人我要加锁了，所以意向锁并不是一个真正物理意义上的锁。

意向锁和共享锁、排它锁相结合，就有了意向共享锁和意向排它锁。

意向共享锁即你希望获得一个共享锁。

意向排它锁即你希望获得一个排它锁。

注意，意向锁的意向强调的就是你想要拿到这个锁，但是你最终能否拿到这个锁，是不确定的。

在 MySQL 里面，使用意向锁的典型场景是在增删改查的时候，对表结构定义加一个意向共享锁，防止在查询的时候有人修改表结构。而在修改表结构的时候，则会加一个意向排它

锁。这也就是修改表结构的时候会直接阻塞掉所有的增删改查语句的原因。**使用意向锁能够提高数据库的并发性能，并且避免死锁问题。**

记录锁、间隙锁和临键锁

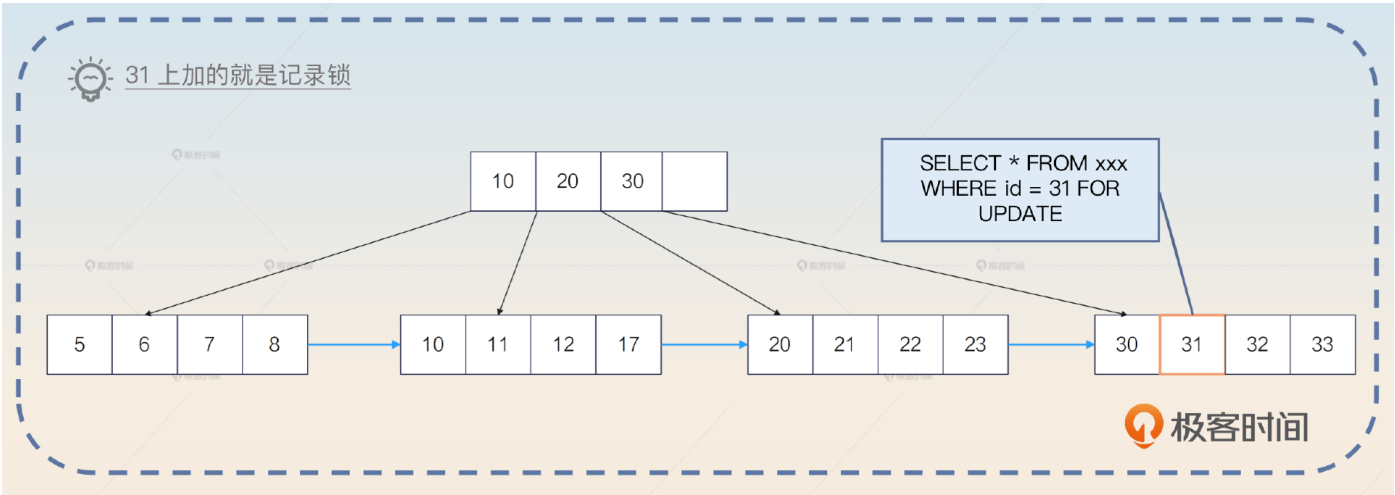
这是面试中最难理解的三个概念，而且要是面试官对细节非常了解，那么你就很容易挂在这一个部分。但是反过来说，如果你能将这部分的细节都说清楚，本身就是一个很大的亮点了。

记录锁

记录锁是指锁住了特定的某一条记录的锁。例如这样一条语句：

```
SELECT * FROM your_tab WHERE id = 31 FOR UPDATE
```

在你使用了主键作为查询条件，并且是相等条件下，将只命中一条记录。这一条记录就会被加上记录锁。



但是如果查询条件没有命中任何记录，那么就不会使用记录锁，而是使用间隙锁。又或者你使用了唯一索引作为条件，比如说在 user 表里面在列 email 上有一个唯一索引。

```
SELECT * FROM your_tab WHERE email='your_email' FOR UPDATE
```

那么这条查询语句此时也是使用了记录锁。类似地，如果 email= 'your_email' 这条记录不存在，那么会变成一个间隙锁。

举个例子，如果数据库中只有 id 为 (1, 4, 7) 的三条记录，也就是 id= 3 这个条件没有命中任何数据，那么这条语句会在 (1, 4) 加上间隙锁。所以你可以看到，**在生产环境里面遇到了未命中索引的情况，对性能影响极大。**

我这里稍微解释一下，实际上MySQL本身是加上临键锁的，但是临键锁本身是由间隙锁和记录锁合并组成的，所以这里我就先用间隙锁来描述了。

间隙锁

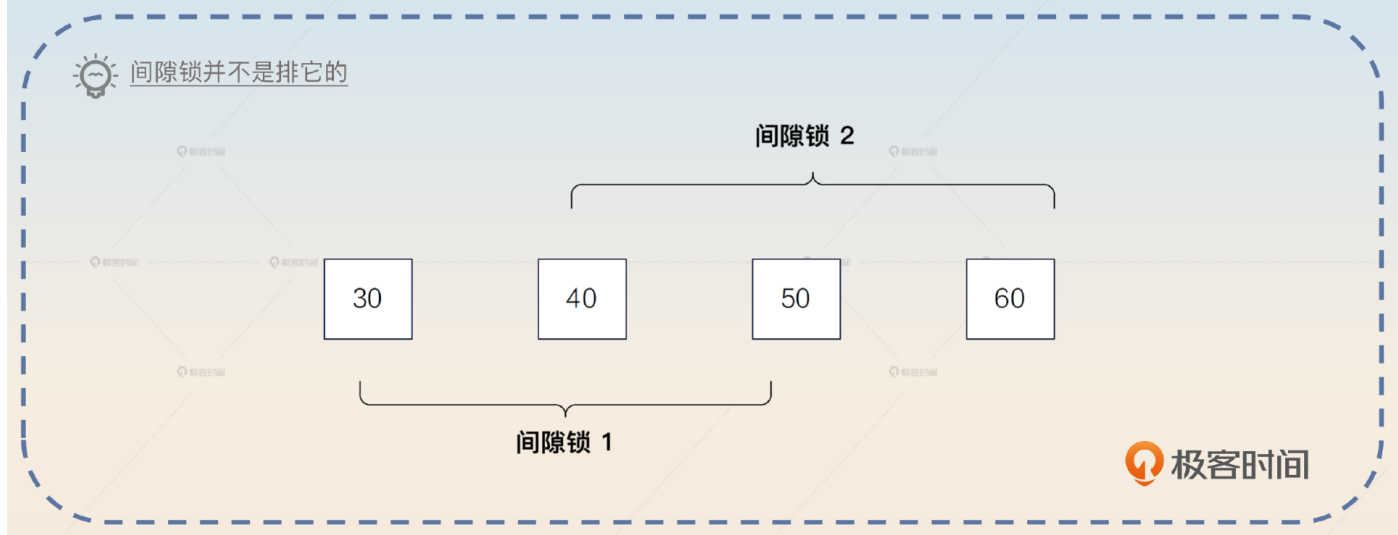
间隙锁是锁住了某一段记录的锁。直观来说就是你锁住了一个范围的记录。比如说你在查询的时候使用了 <、<=、BETWEEN 之类的范围查询条件，就会使用间隙锁。

```
SELECT * FROM your_tab WHERE id BETWEEN 50 AND 100 FOR UPDATE
```

间隙锁会锁住 (50, 100) 之间的数据，而 50 和 100 本身会被记录锁锁住。类似地，<= 这种查询你也可以认为 = 的那个值会被记录锁锁住。

如果你的表里面没有 50，那么数据库就会一直向左，找到第一个存在的数据，比如说 40；如果你的表里面没有 100，那么数据库就会一直向右，找到第一个存在的数据，比如说 120。那么使用的间隙锁就是 (40, 120)。如果此时有人想要插入一个主键为 70 的行，是无法插入的，它需要等这个 SELECT 语句释放掉间隙锁。

间隙锁我们一般都说两边都是开的，即端点是没有被间隙锁锁住的。**记录锁和记录锁是排它的，但是间隙锁和间隙锁不是排它的。**也就是说两个间隙锁之间即便重叠了，也还是可以加锁成功的。



临键锁

临键锁（Next-Key Locks）是很独特的一种锁，直观上来说可以看做是一个记录锁和间隙锁的组合。也就是说**临键锁不仅仅是会用记录锁锁住命中的记录，也会用间隙锁锁住记录之间的空隙**。临键锁和数据库隔离级别的关系最为紧密，它可以解决在可重复读隔离级别之下的幻读问题。

间隙锁是左开右开，而临键锁是**左开右闭**。还是用前面的例子来说明。如果 id 只有 (1, 4, 7) 三条记录，那么临键锁就将 (1, 4] 锁住。

小结一下

我这里依旧给你准备了简洁版的记忆口诀，让你用来判断使用的是记录锁、间隙锁还是临键锁。

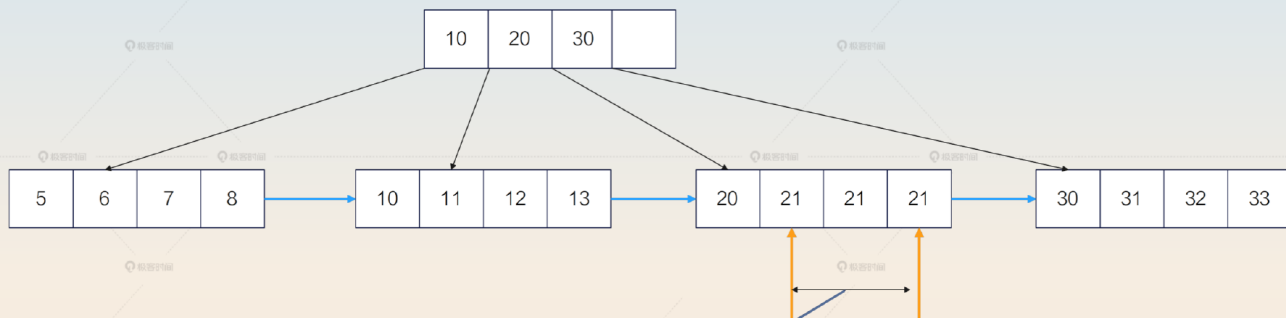
遇事不决临键锁。你可以认为，全部都是加临键锁的，除了下面两个子句提到的例外情况。

右边缺省间隙锁。例如你的值只有 (1, 4, 7) 三个，但是你查询的条件是 WHERE id < 5，那么加的其实是间隙锁，因为 7 本身不在你的条件范围内。

等值查询记录锁。这个其实针对的是主键和唯一索引，普通索引只适用上面两条。



普通索引不会使用记录锁



普通索引因为值可能相同，
所以即使是等值查询，
也不会使用记录锁

极客时间

面试准备

为了更好地面试锁，你需要在公司内部收集一些信息。

公司出现过的死锁，包含排查过程、解决方案。

其他锁使用不当的场景，比如因为锁使用不当造成的一些性能问题。

收集至少一个使用乐观锁的场景，并且看看相关的 SQL 是怎么写的，做到心中有数。

收集公司内使用悲观锁的场景，并且尝试使用乐观锁来进行优化。

这些案例非常重要，如果你自己没有亲自遇到，也要找同事问清楚，或者在互联网上搜集一些案例来实际看看锁的应用。

类似索引，面试官可能会直接写一个 SQL 语句，问你可能加什么锁。这种问题总体上还是偏难的，你在准备面试的时候可以刻意练习一下。

在主键或唯一索引上使用等值查询，例如 `WHERE email = 'abc@qq.com'`，区分记录存在与不存在两种情况。

在主键或唯一索引上使用范围查询，例如 `WHERE email >= 'abc@qq.com'`。

在普通索引上使用等值查询。

在普通索引上使用范围查询。

执行查询，但是该查询不会使用任何索引。

不管你怎么回答，都要记得强调间隙锁和临键锁是在可重复读的隔离级别下才有效果的。

在面试中，如果你非常熟悉 MySQL 锁，那么你可以抓住机会展示一下。

我觉得我对 MySQL 的锁机制还是非常了解的，只要您写出来 SQL，告诉我表结构定义，我就可以告诉您这个语句会加什么锁，会不会阻塞别的语句。

此外，你和面试官在聊到一些话题的时候，你可以尝试把话题引导到锁机制上。

索引：MySQL 的 InnoDB 引擎是借助索引来实现行锁的。

性能问题：锁使用不当引起的性能问题。

乐观锁：比如说原子操作中的 CAS 操作，你可以借助 CAS 这个关键词，聊一聊在 MySQL 层面上怎么利用类似的 CAS 操作来实现一个乐观锁。

语言相关的锁：比如说 Go 语言的 mutex 和 Java 的 Lock，都可以引申到数据库的锁。

死锁：聊一聊公司的数据库死锁案例。

整体来说，数据库锁优化在整个性能优化领域里面也算是一个比较高级的点了，很有竞争优势，不可错过。

基本面试

锁这边的问题会有非常多的问法。一开始引起锁的话题的时候，面试官可能这么问。

你知道 MySQL 的锁机制吗？

你了解 MySQL 的锁吗？

那么类似的问题你就可以先综合回答，介绍一下 MySQL 里面的五花八门的锁。

MySQL 里面的锁机制特别丰富，这里我以 InnoDB 引擎为例。首先，从锁的范围来看，可以分成行锁和表锁。其次，从排它性来看，可以分成排它锁和共享锁。还有意向锁，结合排它性，就分为排它意向锁和共享意向锁。还有三个重要的锁概念，记录锁、间隙锁和临键锁。记录锁，是指锁住某条记录；间隙锁，是指锁住两条记录之间的位置；临键锁可以看成是记录锁与间隙锁的组合

情况。

还有一种分类方法，是乐观锁和悲观锁。那么在数据库里面使用乐观锁，本质上是一种应用层面的 CAS 操作。

紧接着，你不要深入去解释各种锁，而是先从大方向上解释清楚锁的根本特性：**锁是和索引、隔离级别密切相关的。**

在 MySQL 的 InnoDB 引擎里面，锁和索引、隔离级别都是有密切关系的。在 InnoDB 引擎里面，锁是依赖于索引来实现的。或者说，锁都是加在索引项上的。因此，如果一个查询用了索引，那么会用行锁，如果没用到任何索引，那么就会用表锁。此外，在 MySQL 里面，间隙锁和临键锁是只工作在可重复读这个隔离级别下的。

然后你就等着面试官进一步追问细节。正常情况下，他可能问下面这些问题。

某一种锁的具体含义。

某一种锁的使用场景，这里稍微注意一点意向锁就可以，其他的都比较简单。

怎么在数据库里面使用乐观锁，或者你用乐观锁解决过什么问题？

你有没有优化过锁，或者解决过死锁？

详细介绍记录锁、间隙锁和临键锁，也有可能问你 MySQL 在可重复读的隔离级别下，会不会有幻读问题？

这里，你可以详细分析记录锁、间隙锁和临键锁，不过如果想要百分之百征服面试官，你还是需要用一些实际的锁优化的案例来证明你的能力。

亮点方案

这里我先说一个最简单的锁优化的方案。我们前面提到，MySQL 的锁是依赖索引机制来实现的。而如果查询没有使用任何索引，就会使用表锁。那么很显然最简单的方案就是给这种查询创建一个索引，避免使用表锁。

你可以这么回答，关键词是**缺索引**。

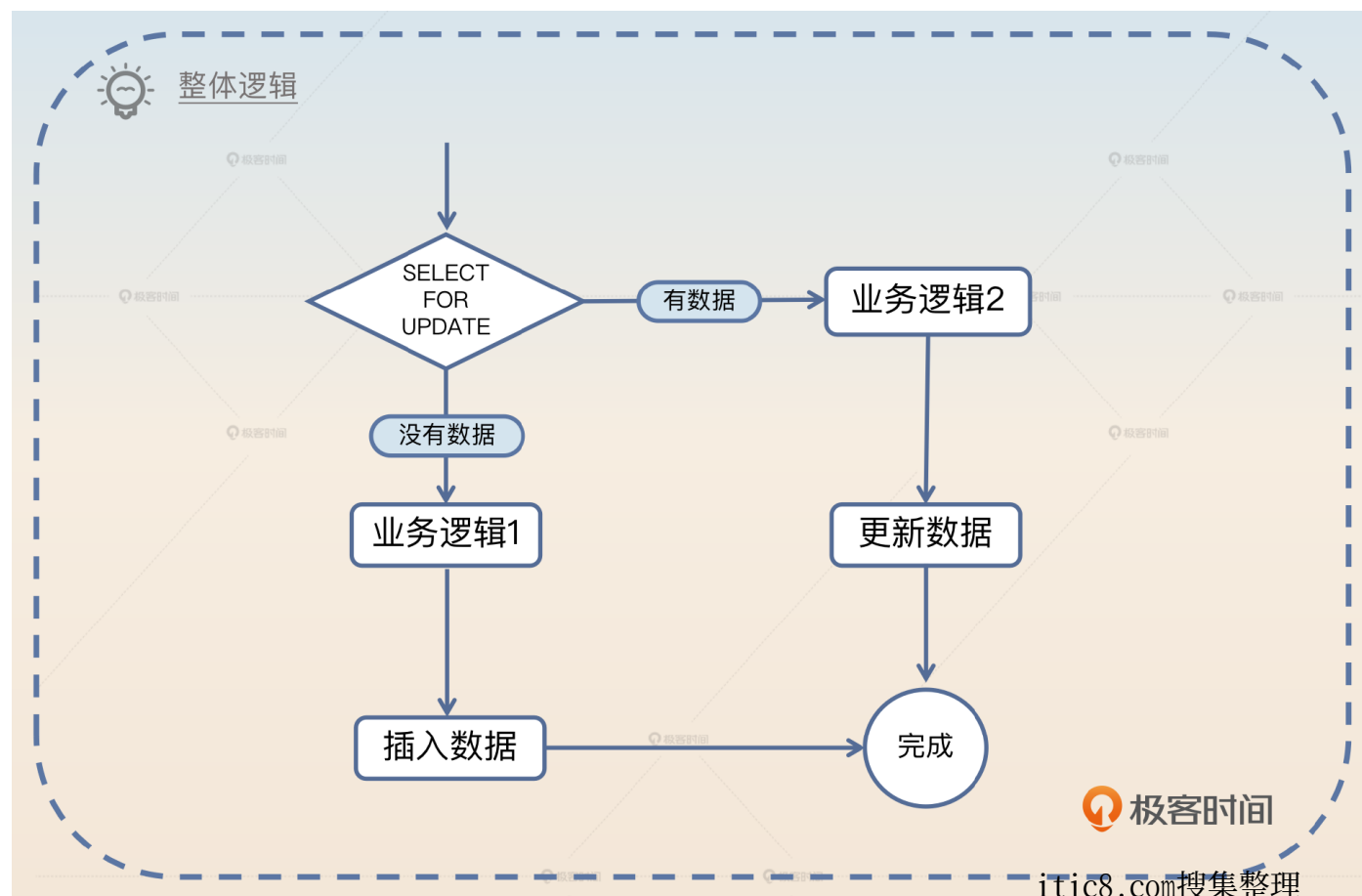
早期我发现我们的业务有一个神奇的性能问题，就是响应时间偶尔会突然延长。后来经过我们排查，确认响应时间是因为数据库查询变慢引起的。但是这些变长的查询，SQL 完全没有问题，我用 EXPLAIN 去分析，都很正常，也走了索引。

直到后面我们去排查业务代码的提交记录，才发现新加了一个功能，这个功能会执行一个 SQL，但是这个 SQL 本身不会命中任何索引。于是数据库就会使用表锁，偏偏这个 SQL 因为本身没有命中索引，又很慢，导致表锁一直得不到释放。结果其他正常的 SQL 反而被它拖累了。最终我们重新优化了这个使用表锁的 SQL，让它走了一个索引，就解决了这个问题。

不过这个方案说起来还是太简单，我们需要一些更复杂的方案来打动面试官。那么接下来我再给你提供两个稍微复杂一些的方案：临键锁引发的死锁和弃用悲观锁。

临键锁引发的死锁

在一个业务中，有一个场景是先从数据库中查询数据并锁住。如果这个数据不存在，那么就需要执行一段逻辑，计算出一个数据，然后插入。如果已经有数据了，那么就将原始的数据取出来，再利用这个数据执行一段逻辑，计算出来一个结果，执行更新。



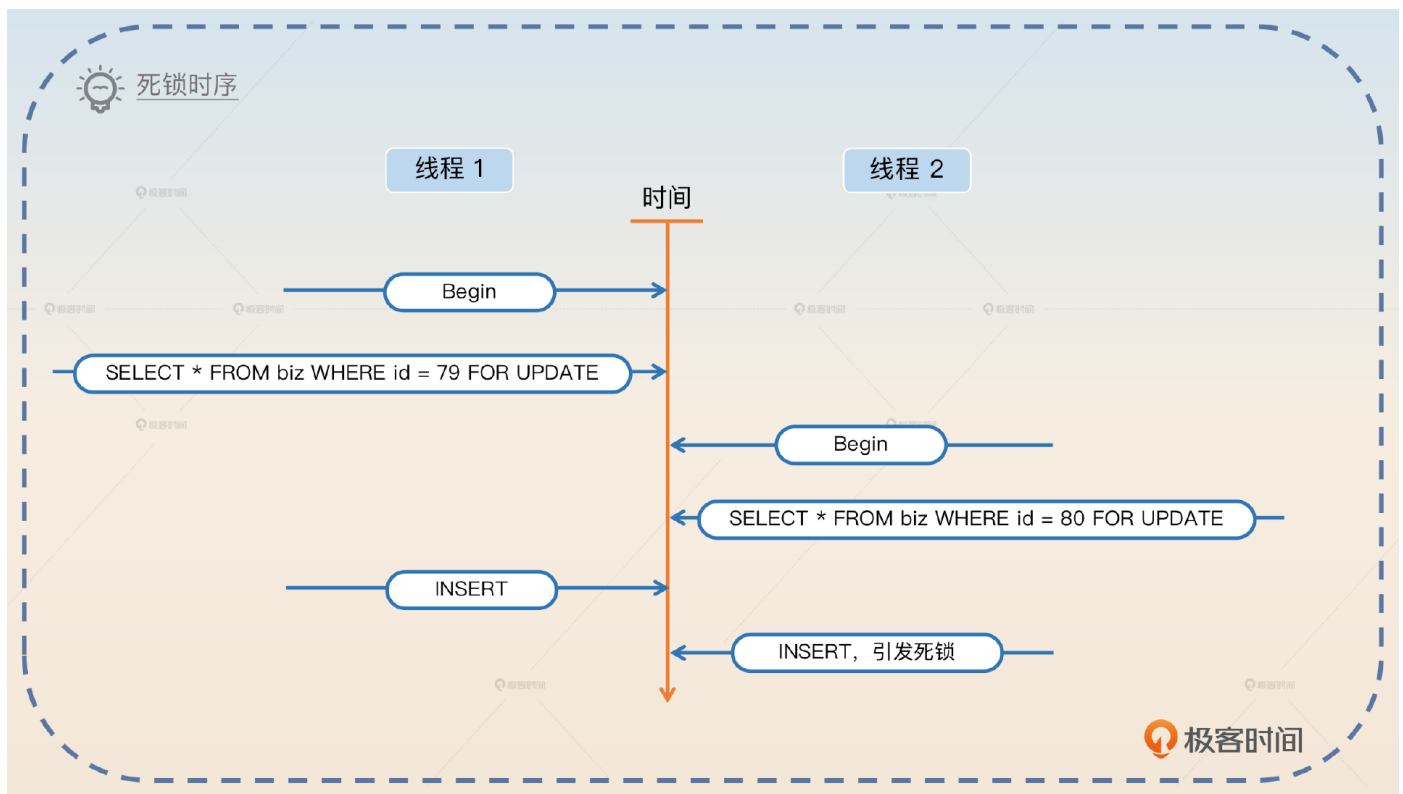
因为两段运算逻辑不同，所以不能简单地使用 INSERT ON DUPLICATE 的语句来取代。

只看没有数据的逻辑，在计算之后插入新数据，如果用伪代码来描述，就是下面这样的。

```
BEGIN;  
SELECT * FROM biz WHERE id = ? FOR UPDATE  
// 中间有很多业务操作  
INSERT INTO biz(id, data) VALUE(?, ?);  
COMMIT;
```

看起来是不是没有任何问题？实际上，这个地方会引起**死锁**。

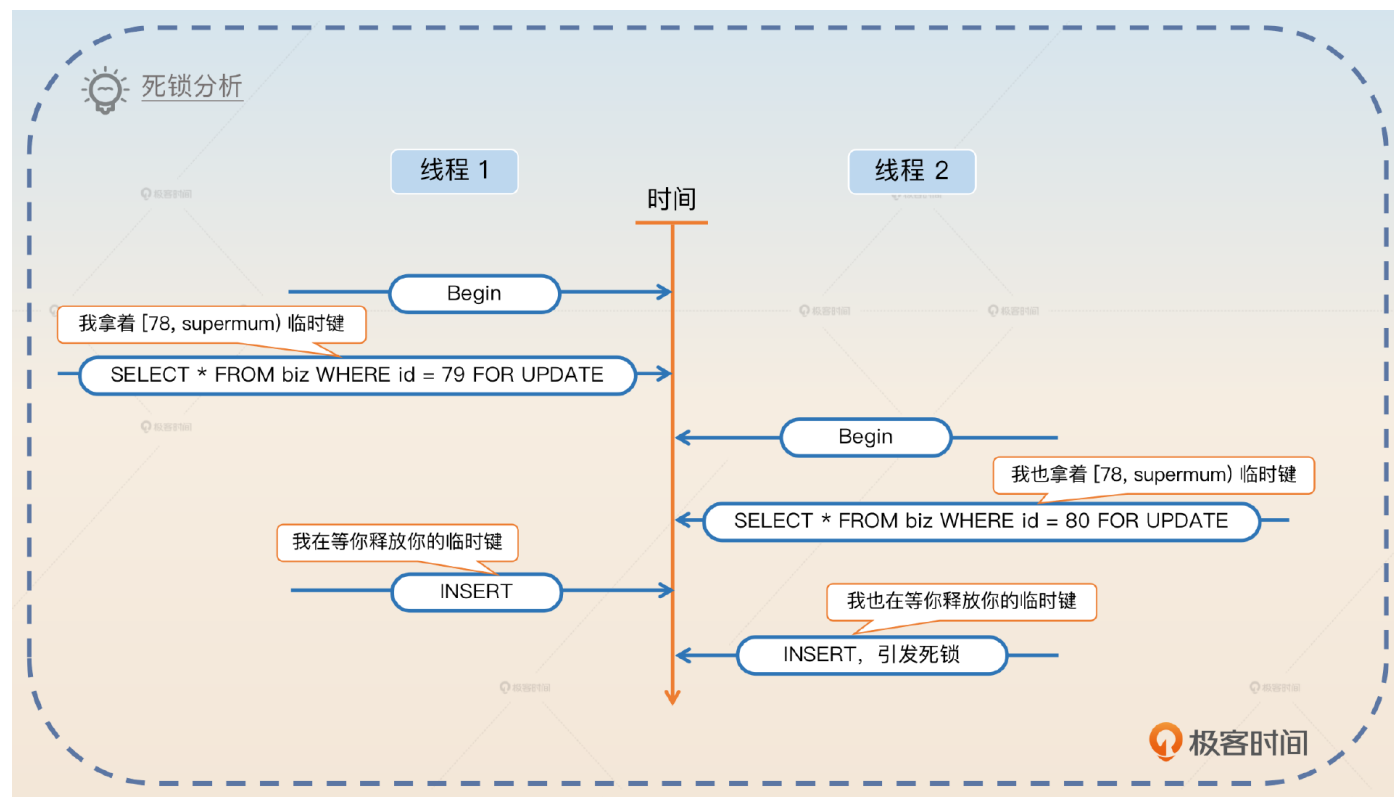
假设说现在数据库中 ID 最大的值是 78。那么如果两个业务进来，同时执行这个逻辑。一个准备插入 id=79 的数据，一个准备插入 id = 80 的数据。如果它们的执行时序如下图，那么你就会得到一个死锁错误。



[40001][1213] Deadlock found when trying to get lock; try restarting transaction

造成死锁的原因也很简单。在线程 1 执行 SELECT FOR UPDATE 的时候，因为 id=79 的数据不存在，所以实际上数据库会产生一个(78, supremum] 的临键锁。类似地，线程 2 也会产生一个(78, supremum] 临键锁。

当线程 1 想要执行插入的时候，它想要获得 id = 79 的行锁。当线程 2 想要执行插入的时候，它想要获得 id = 80 的行锁，这个时候就会出现死锁。因为线程1 和线程 2 同时还在等着对方释放掉持有的间隙锁。



从理论上来说，解决方案有三种。

不管有没有数据，先插入一个默认的数据。如果没有数据，那么会插入成功；如果有数据，那么会出现主键冲突或者唯一索引冲突，插入失败。那么在插入成功的时候，执行以前数据不存在的逻辑，但是因为此时数据库中有数据，所以不会使用间隙锁，而是使用行锁，从而规避了死锁问题。

调整数据库的隔离级别，降低为已提交读，那么就没有间隙锁了。这种解决方案也可以，而且你可以将话题进一步引申到 MVCC 中。

放弃悲观锁，使用乐观锁。这也是我提供的亮点方案。

你可以通过一个案例来说明，关键词是**临键锁**。

早期我优化过一个死锁问题，是临键锁引起的。业务逻辑很简单，先用 SELECT FOR UPDATE 查询数据。如果查询到了数据，那么就执行一段业务逻辑，然后更新结果；如果没有查询到，那么就执行另外一段业务逻辑，然后插入计算结果。

那么如果 SELECT FOR UPDATE 查找的数据不存在，那么数据库会使用一个临键锁。此时，如果有两个线程加了临键锁，然后又希望插入计算结果，那么就会造成死锁。

我这个优化也很简单，就是上来先不管三七二十一，直接插入数据。如果插入成功，那么就执行没有数据的逻辑，此时不会再持有临键锁，而是持有了行锁。如果插入不成功，那么就执行有数据的业务逻辑。

此外，还有两个思路。一个是修改数据库的隔离级别为 RC，那么自然不存在临键锁了，但是这个修改影响太大，被 DBA 否决了。另外一个思路就是使用乐观锁，不过代码改起来要更加复杂，所以就没有使用。

你这么回答的话，面试官接下来就可能追问你隔离级别的事情，或者问你乐观锁的细节，那么你就有机会继续展示了。

弃用悲观锁

很多人为了省事会在开发的时候直接使用悲观锁。最为典型的例子就是在事务里面存在 SELECT ... FOR UPDATE 的语句，而后会紧跟着一个 UPDATE 语句。

伪代码：

```
// 开启事务
Begin()
// 查询到已有的数据 SELECT * FROM xxx WHERE id = 1 FOR UPDATE
data := SelectForUpdate(id)
newData := calculate(data) // 一大通计算

// 将新数据写回去数据库 UPDATE xxx SET data = newData WHERE id =1
Update(id, newData)
Commit()
```


那么这一类代码可以考虑将整个事务都去掉，纯粹依赖 CAS 操作。

```
for {
    // 查询到已有的数据 SELECT * FROM xxx WHERE id = 1
    data := Select(id)
    newData := calculate(data) // 一大通计算

    // 将新数据写回去数据库
    // UPDATE xxx SET data = newData WHERE id =1 AND data=oldData
    success := CAS(id, newData, data)
    // 确实更新成功，代表在业务执行过程中没有人修改过这个 data。
    // 适合读多写少的情况
    if success {
        break;
    }
}
```

这里我是直接用的 data 来比较的，实践中也可能是引入了 version 列，或者使用 update_time 来确保数据没有发生变更。

你可以在聊到乐观锁的情况下，使用这个案例。

我在入职这家公司之后，曾经系统地清理过公司内部使用悲观锁的场景，改用乐观锁。正常的悲观锁都是使用了 SELECT FOR UPDATE 语句，查询到数据之后，进行一串计算，再将结果写回去。那么改造的方案很简单，查询的时候使用 SELECT 语句直接查询，然后进行计算。但是在写回去的时候，就要用到数据库的 CAS 操作，即 UPDATE 的时候要确认之前查询出来的结果并没有实际被修改过。

一般来说就是 UPDATE xxx SET data = newData WHERE id = 1 AND data = oldData。这种改造效果非常好，性能提升了 30%。当然，并不是所有的悲观锁场景都能清理，还有一部分实在没办法，只能是考虑别的手段了。

这里最后留了一个尾巴，也就是将话题引导到你准备的其他优化锁的案例上。

面试思路总结

这节课我们学习了面试中的重难点问题——锁，锁与索引的关系十分密切，可以说锁是借助索引来实现的。锁的形式五花八门，有乐观锁、悲观锁、行锁、表锁、共享锁、排它锁、意向锁、记录锁、间隙锁和临键锁等多种类型，它们各有用途，这里你要注意分辨它们各自的能力，学会分析什么场景下使用哪一种锁。

此外，我还提供了两个比较复杂的亮点方案：**临键锁引发的死锁、弃用悲观锁**。你在实际面试中可以考虑替换为自己的案例。



最后你来思考两个问题。

你在公司有没有优化过什么锁？可以分享一下你的案例吗？

你有没有使用过乐观锁？用于解决什么问题？相比直接使用悲观锁，你认为代码量有没有变化？

欢迎你把你的答案分享在评论区，也欢迎你把这节课的内容分享给需要的朋友，我们下节课再见！

题外话：面试锦囊

最后我们再来聊点儿题外话，就是面试时的小技巧——能示之不能，不能示之能。在前面很多讲里，我都在回答中都故意点到即止，或者故意留出破绽，就是“能示之不能”，引诱面试官追问。

那么反过来“不能示之能”就是纯粹地唬人。就是说实际上你并不是很了解某一些方面的东西，但是你假装你非常了解、胸有成竹。那么在这种情况下，如果面试官被你的假象迷惑住了，那么他就不会继续追问这方面的东西。

这跟我们的面试流程是有关的。很多人出去面候选者，讲究的是问到候选者无话可说。那么既然你表现得一副我还有很多话说，快来问我的样子，他们自然不会问你了。

不过“不能示之能”策略要慎用。一场面试可一不可二，可二千万别三。因为你装多了，面试官就会好奇你怎么什么都知道，或者激起了逆反心理，深挖下来你就挡不住了。

但是“能而示之不能”就可以随使用。尤其是一些公司有所谓的“压薪面”。就是本身你已经通过了基本的技术面试，但是公司不想给你高薪，就会故意面你一些很难的问题，打击你，这样好压价。那么如果你能够在压薪面里面将面试官引导到你熟悉的领域，立于不败之地，那么接下来谈薪酬就更有主动权。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 11 | SQL优化：如何发现SQL中的问题？

下一篇 13 | MVCC协议：MySQL 在修改数据的时候，还能不能读到这条数据？

精选留言 23



penbox

1689142653

《乐观锁与悲观锁》部分是不是有个BUG？

“悲观锁是指在写入数据时直接加锁”，我觉得应该是“悲观锁是指在获取数据时直接加锁”

作者回复 可能是我这里因为要突出和上一条乐观锁的对比，让你产生了误解。应该是，悲观锁不管读写都要直接加锁。



子休

1689757711

研究过一段时间锁的机制。举几个点，来分享一下如何理解临键锁。

首先这个名词“临键”非常拗口难记，但是大家仔细观察它的英文“next-key lock”，就可以发现一些名堂，我个人认为，所谓next-key的意思就是下一个索引的意思，也就是锁住查询结果中的最大索引值与下一个索引值之间的区域。这就意味着，临键锁锁住的最后的那个区间，是当前命中的索引最大值到下一个索引的区间，如果没有下一个索引，那就是锁住了剩下所有区间。结合本文的例子，大家就容易理解了。

例子1：如果 id 只有 (1, 4, 7) 三条记录，你查的是 where id = 3，虽然没有查到记录，但是由于innodb中RR级别会使用临键锁，于是临键锁要开始锁区间了，但是锁哪里呢？这时候就要理解“临键”了，就是“next-key”，那么“next-key”是谁呢，根据查询条件ID=3，那么3后面的下一个索引是几呢？是4！！那么临键锁就将 (1, 4]锁住了。

另外要额外提一句，由于查询条件可能是区间查询，所以临键锁会锁住多个区间。

比如查询条件 id > 1 and id < 6 它就会把(1, 4],[4, 7]这两个区间都锁住。

又比如查询条件是id > 1 and id < 9 它就会把(1, 4],[4, 7],[7, +∞]这三个区间都锁住，换句话说，这种查询也就导致了根本插入不了任何一条记录，因为它把 id 从1到无穷的范围都给锁住了。

为什么要锁住区间？因为要防止幻读出现（幻读就是同一事务里面，同一个sql查询查出来的记录行数不一样。为什么会不一样？因为有别的事务在你执行sql的时候进行了插入，插入到了你的查询条件范围内了，导致你上一次查还好好好的，下一次查就莫名其妙多出来记录了。）。

所以，你想，临键锁把你查询条件范围的区间锁住了，其它事务想往区间里面插数据是不是就不行了？临键锁是根据你的查询条件来锁区间的，这样你在同一事务里反复执行同一条sql查询，是不是就不会出现幻读了。

以上是个人浅见，欢迎指正。

itjc8.com搜集整理

作者回复 赞！我也无力吐槽临键锁这个翻译，但是大家都用我也跟着用。



humor

1689160225

举个例子，如果数据库中只有 id 为 (1, 4, 7) 的三条记录，也就是 id= 3 这个条件没有命中任何数据，那么这条语句会加上间隙锁，而且是在 $(-\infty, 1)$ 、 $(1, 4)$ 、 $(7, +\infty)$ 这些地方都加上间隙锁。所以你可以看到，在生产环境里面遇到了未命中索引的情况，对性能影响极大。

这个查询只会扫描到记录1和4吧，为什么7以后也加了间隙锁呢

作者回复 写错了，我修正一下！感谢指正！



rrbtt

1689131120

之前写代码，事物里面，select语句从来不在后面加for update结尾。这样会有什么问题吗？

作者回复 如果你不加 for update 的话，正常来说是根据隔离级别，用 MVCC 的机制来读取数据。



Jason Ding

1693813113

Next-Key Locks 没有完全解决幻读问题

作者回复 又是怎么个说法？

To prevent phantoms, InnoDB uses an algorithm called next-key locking that combines index-row locking with gap locking.

<https://dev.mysql.com/doc/refman/8.0/en/innodb-next-key-locking.html>



Geek8004

1693754664

意向共享锁和MDL读锁；意向排他锁和mdl写锁什么关系,等价吗？ itic8.com搜集整理

作者回复 不是。意向锁的意思是我将来要拿一个锁，MDL 是你已经拿到了锁。所以我的理解就是，如果你要拿MDL 读锁，你现有一个意向共享锁。写锁也是类似的。



牙小木

1691650213

哈哈，实在又实用，

作者回复 谢谢！



牙小木

1691640774

提个醒：查询的索引含有唯一属性的时候，Next-Key Lock 会进行优化，将其降级为Record Lock，即仅锁住索引本身，但是普通索引没有唯一属性，就会升级为gap了

作者回复 是的。不过其实我觉得与其理解为优化，不如说如果我是一个设计者，我也会直接咔嚓一个记录锁锁住，因为在这种情况下我能准确推断出来哪些需要被锁住。



牙小木

1691637900

<https://dev.mysql.com/doc/refman/5.7/en/innodb-locking.html>



Geek_9af983

1690713463

老师宁好呀，什么情况下select 需要加for update呀，我们项目中从来没这么写过

作者回复 你需要显式锁住某一行，不想别人修改的时候。你们不用说明你们可能用了乐观锁、分布式锁之类的东西。

不用是好事，哈哈。



胡月

1690500831

我做过的项目隔离级别都是读已提交。可重复读隔离级别没用到过，有什么场景必须用可重复读吗？

作者回复 问倒我了，我自己没有遇到过。我认为在互联网业务里面应该没啥必须得场景，但是金融那边就不太确定。



Geek_18dfaf

1690334947

老师，为什么间隙锁不做排他操作呢

作者回复 间隙锁要是排它，性能就太差了。你想查询一个数据，结果因为间隙锁有重合，导致查询不了，你肯定不能忍。



Geek_18dfaf

1690333176

间隙锁不是排它的，那间隙锁中间插入数据不会出问题么

作者回复 应该说，你可以加很多间隙锁，间隙锁之间是可以重合的，然后这个时候你可以读数据。但是，写操作是不行的。



陈斌

1689779807

意向锁除了变更表结构的例子，还是有其他的例子吗？毕竟意向锁有这么多的好处：使用意向锁能够提高数据库的并发性能，并且避免死锁问题。

SQL语句中怎么手动使用意向锁？

作者回复 遗憾的是，你并不能手动指定用意向锁，这个是 MySQL 来控制的。



陈斌

1689779618

我这里依旧给你准备了简洁版的记忆口诀，让你用来判断使用的是记录锁、间隙锁还是临键锁。

遇事不决临键锁。你可以认为，全部都是加临键锁的，除了下面两个子句提到的例外情况。右边缺省间隙锁。例如你的值只有 (1, 4, 7) 三个，但是你查询的条件是 WHERE id < 5，那么加的其实是间隙锁，因为 7 本身不在你的条件范围内。（这里的7是不是写错了，应该是5）

等值查询记录锁。这个其实针对的是主键和唯一索引，普通索引只适用上面两条。

你给的第二个原则是不是说：如果是有比较的查询条件 (>, >=, <, <=)，加锁都是加的间隙锁，记录有1, 3, 7，并且id是唯一索引情况下：id < 5，加的是 (-inf, 5) 还是 (-inf, 3)的间隙锁？如果是(-inf, 3)的间隙锁有4插入进来不就导致加锁失败？

作者回复 不是，在 1, 4, 7 里面，< 5 这个条件，你会发现第一个比 5 大的是 7，但是 7 不在你的条件范围里，是条件范围，所以是 -inf, 7 的间隙锁。注意，你加锁的那个锁，必然是和你真的有的一个数据相关的，因为锁本身是借助索引来实现的，也就是你需要有对应的索引项。



陈斌

1689778970

能讲讲加锁的原理吗，例如我可以利用该原理分析出 在普通索引上使用等值查询和在普通索引上使用范围查询怎么加锁？

看完文章之后，你在文章中提出的问题，给出一个SQL要分析如何加锁还是不会。

作者回复 加什么锁我应该讲得比较清晰了。如果你觉得还是没看懂，那么可以看看林老师的 mysql 45 讲，里面讲得更加详细。

坦白说，你现在给我一个，我也很难分析出来。我每次都是面试前突击一把，趁着还记得赶紧出去面试。平时工作中分析，也是打开自己的笔记对着来分析。



陈斌

1689778608

直接使用JPA框架插入一条记录，主键id是UUID自动生成的，这个SQL语句会加锁吗？

作者回复 uuid 生成的时候，MySQL 肯定没有感知，所以不会加锁。但是你插入的时候，mysql 就要锁住这个 ID 对应的行。



gevin

1689730411

要保证业务结果的正确，在业务代码里加锁，或在数据库层加锁，应该都可以的吧？选择在哪儿实现锁的逻辑，实践中有没有啥常用的指导原则？

作者回复 一般在追求并发和性能的程序里面，要尽可能减轻数据库的负担，所以原则都是能在应用层面解决的，就在应用层面解决。



xyu

1689654497

大明老师，我这里有个疑问：

针对“一般来说就是 UPDATE xxx SET data = newData WHERE id = 1 AND data = oldData。这种改造效果非常好，性能提升了 30%”这句话，乐观锁对悲观锁的改造能提升性能，是针对读多写少的场景吧？如果是写多的场景，基于悲观锁的逻辑性能是否可能会更好一点？

谢谢！

作者回复 是的！悲观锁适合写多读少的，写频繁用乐观锁反而会使得性能更差。



peter

1689170174

请教老师几个问题：

Q1：Java中有协程池吗？

模拟面试（一）中提到了协程池，一般都是用Java开发，Java中没有协程啊。

Q2：这几种锁，具体是怎么定义的？以Java代码为例，锁是在SQL语句中指定？还是在Java代码中指定？比如悲观锁，是在某个地方加一个关键字“BeiGuan”吗？

Q3：MySQL有两个引擎，InnoDB和MyISAM，怎么选？

Q4：记录锁等于行锁吗？都是锁住一行啊。

Q5：“临键锁”，第一个字应该是“邻”吗？

Q6：会讲事务吗？希望能讲讲事务。

Q7：共享锁，两个线程都加锁的话，哪个有效？

作者回复 1. JAVA 是没有的，因为JAVA 没有协程概念。你在这里用线程池和连接池来替代。

2. 悲观锁就是你正常用的锁。你要区别你是在应用上加锁还是在数据库上加锁。应用上加锁，就是你自己用 Lock；SQL 上加锁，一般不需要你管，SELECT 语句你就需要用 FOR UPDATE

3. 无脑用 InnoDB，后面等你深入研究了再说。

4. 实际上记录锁和行锁只是从不同的角度来看，大部分情况下，就是一把锁的不同名称。

来自公众号：码农收集整理

5. 目前我看大部分中文翻译都是用 临键锁
6. 后面内容有，不要急。
7. 都有效。